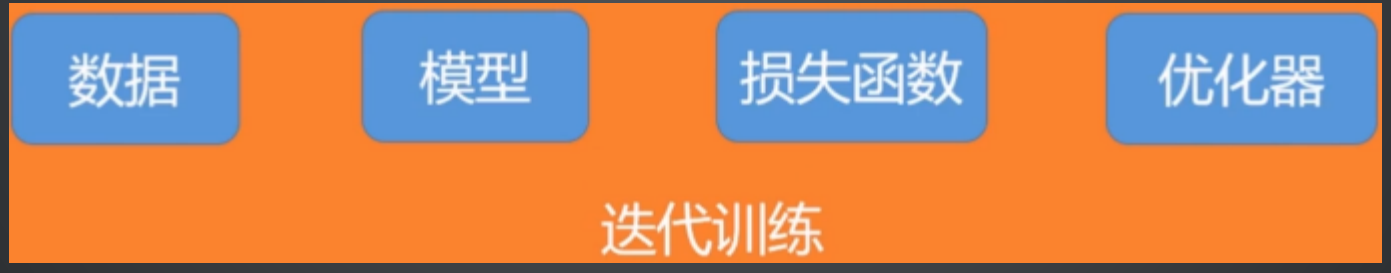

大数据与人工智能
自动化学院

Chapter03.06 模型的权值初始化与损失函数介绍

3.6-1 - 神经网络的“起点”与“裁判”：权值初始化与损失函数



- **前情提要**: 上一节，我们学会了如何用不同的“积木”（如卷积层、全连接层）搭建起一个神经网络的“骨架”。
- 现在，我们要讨论两个关键问题：
    1. **起点在哪里？** 模型刚开始训练时，内部的参数（权重）应该设置成多少？这就是**权值初始化**。
    2. **好坏谁来评判？** 模型预测得准不准，谁来打分？这就是**损失函数**，它扮演着“裁判”的角色。

- **内容:**
    1. **第一站：设置一个好的起点 (权值初始化)**
        - **为什么起点很重要？** 了解一个常见问题：“梯度消失”与“梯度爆炸”，它们就像模型训练路上的“沼泽”和“悬崖”。
        - **如何找到好起点？** 学习两种主流方法：Xavier方法和Kaiming方法，它们是前辈们总结出的宝贵经验。
    2. **第二站：聘请一位好裁判 (损失函数)**
        - **什么是损失函数？** 它是衡量模型“预测”与“现实”之间差距的标尺。差距越大，损失（分值）越高。
        - **认识最常用的裁判：** 深入了解“交叉熵损失函数”，它是分类任务中最常见的裁判。

3. **总结与梳理:** 将所有知识点串起来，形成清晰的知识地图。

2.权值初始化

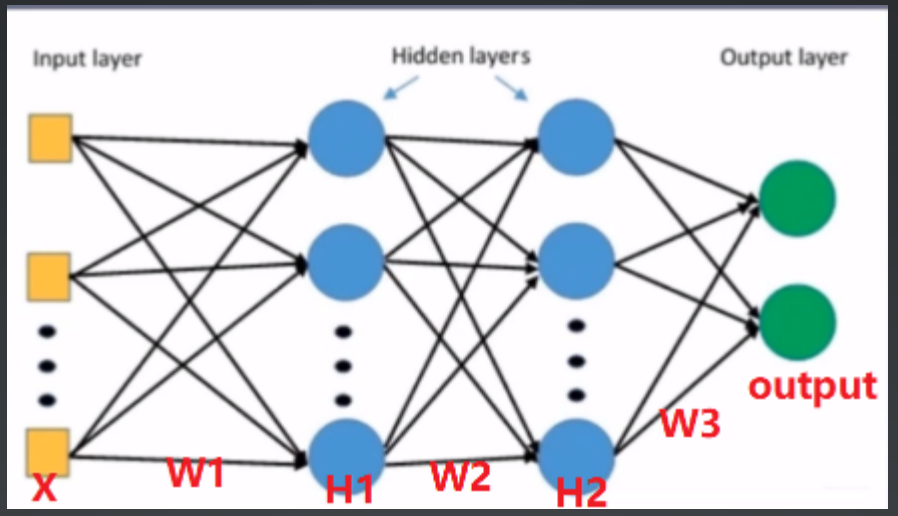
搭建好网络模型后，我们必须给其中的参数（权重）一个初始值。这个初始值非常关键：

- **好的开始是成功的一半：** 如果初始值设置得好，模型就像站在山顶往下跑，能很快找到最优解，训练速度飞快。
- **糟糕的开始会导致“迷路”：** 如果初始值设得不好，模型可能一开始就跑偏了，需要绕很多弯路。更糟糕的是，它可能会引发“梯度消失”或“梯度爆炸”，导致模型直接“罢工”，完全无法训练。

先了解一下什么是梯度消失和梯度爆炸现象

2.1梯度的消失和爆炸

我们以上一篇的一个图来看一下梯度消失和爆炸现象



看上面这个图， 假设我们要算 W2的梯度，根据复合函数求导法则：我们根据链式法则应该是下面这样：

$$\Delta W_2 = \frac{\partial Loss}{\partial W_2} = \frac{\partial Loss}{\partial out} * \frac{\partial out}{\partial H_2} * \frac{\partial H_2}{\partial w_2}$$

$$H_2 = H_1 * W_2$$

$$\begin{aligned}\Delta W_2 &= \frac{\partial Loss}{\partial W_2} = \frac{\partial Loss}{\partial out} * \frac{\partial out}{\partial H_2} * \frac{\partial H_2}{\partial w_2} \\ &= \frac{\partial Loss}{\partial out} * \frac{\partial out}{\partial H_2} * H_1\end{aligned}$$

- 会发现 W 2 梯度的求解过程中会用到上一层神经元的输出值 H 1
  - 如果 H 1 的输出值非常小，那么 W 2的梯度也会非常小
  - 如果H1很大，那么W2的梯度也会很大

$$H_1 \to 0 \Rightarrow \Delta W_2 \to 0$$
$$H_1 \to \infty \Rightarrow \Delta W_2 \to \infty$$

梯度的传播

- 在反向传播中，梯度通过链式法则计算，即每一层的梯度是后续层梯度的乘积。
- 如果某一层的梯度非常小（或非常大），这种连乘效应会导致前面层的梯度迅速减小到接近零（或急剧增大）
- 级联层的梯度如果很小，有可能出现梯度消失。
- 级联层的梯度非常大的时候，当然也就会发生梯度爆炸

- 一旦发生梯度消失或者爆炸， 就会导致模型无法训练
- 如果想避免这个现象，我们就得**控制网络输出层的一个尺度范围，也就是不能让它太大或者太小**。
- 那么我们怎么控制这个网络输出层的尺度呢?
    - 通过合理的初始化权重

随堂作业

不恰当的权值初始化可能会导致网络在训练过程中出现 _____ 或 _____ 的问题，前者指梯度信号过小导致网络前端无法学习，后者指梯度信号过大导致训练崩溃。

答案: 梯度消失, 梯度爆炸

- **解析**: 这是不恰当初始化最直接的两个后果。当网络层数加深，梯度在反向传播中连乘，如果每一步都小于1则趋近于零（消失），如果都大于1则趋近于无穷大（爆炸）。

我们下面从代码切入，进行理解：

- 建立一个100层的多层感知机，每一层256个神经元，我们使用学习过的ModuleList进行建立：

```
class MLP(nn.Module):
    def __init__(self, neural_num, layers):
        super(MLP, self).__init__()
        self.linears = nn.ModuleList([nn.Lin
        self.neural_num = neural_num
    # 正向传播
    def forward(self, x):
        for (i, linear) in enumerate(self.li
            x = linear(x)

            print("layer:{} std:{}".format(
```

- 这说明网络出现了问题，导致后面输出的值太大了
- 反向传播的时候，这些权重根本就没法进行更新，会发生梯度爆炸现象。

bigAI-03-06-20241125.py launch.json

01_PyTorch_Practice > lesson4 > bigAI-03-06-20241125.py > ...

```
57  if flag:
58      layer_nums = 100
59      neural_nums = 256
60      batch_size = 16
61
62      net = MLP(neural_nums, layer_nums)
63      net.initialize()
64
65      inputs = torch.randn((batch_size, neural_nums))  # normal: mean=0, std=1
66
67      output = net(inputs)
68      print(output)
69
70  # ========================================== calculate gain ========================
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS VARIABLES

```
layer:23, std:1.6921342454001848e+25
layer:25, std:2.1257213262324592e+27
layer:26, std:2.191710967109107e+28
layer:27, std:2.525450570646784e+29
layer:28, std:3.221309179111329e+30
layer:30, std:4.5253540317472455e+32
layer:31, std:4.7150121712384475e+33
layer:32, std:5.36959116472917e+34
layer:33, std:inf
layer:34, std:inf
layer:35, std:nan
output is nan in 35 layers
```

这就是有时候我们在训练网络的时候，最后结果全是nan的原因，这往往可能是权重初始化的不当导致的。

```
tensor([[nan, nan, nan,  ..., nan, nan, nan],
        [nan, nan, nan,  ..., nan, nan, nan],
        [nan, nan, nan,  ..., nan, nan, nan],
        ...,
        [nan, nan, nan,  ..., nan, nan, nan],
        [nan, nan, nan,  ..., nan, nan, nan],
        [nan, nan, nan,  ..., nan, nan, nan]], grad_fn=<ReluBackward0>)
```

- 可是，为啥初始化权重不当了会影响到网络的输出呢?
- 前面提到反向传播的时候：某一线性层的输出，影响该层后面的权重梯度
- 正向传播的时候，每一层初始化的权重肯定要影响这一层的输出

推导一下上面这个过程

下面先进行一个 D(XY)方差的公式推导：

借助三个基本公式：

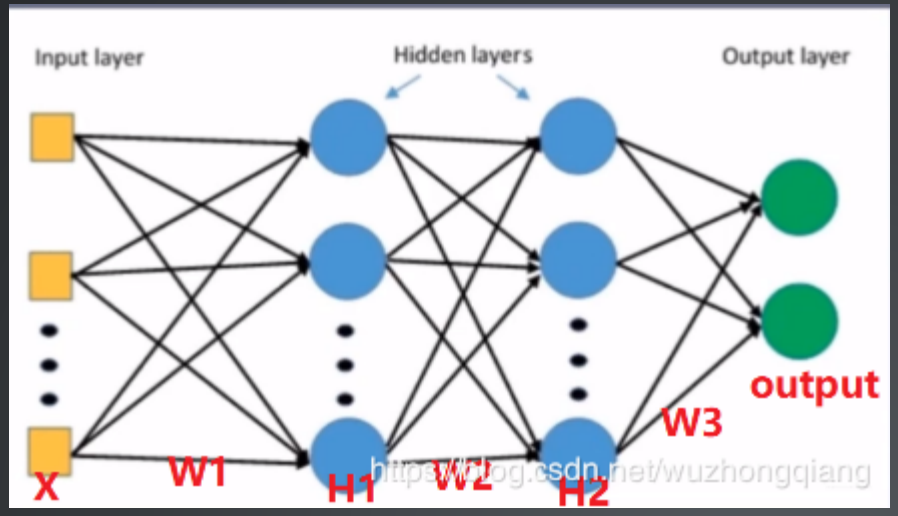
- $E(XY) = E(X)E(Y)$
- $D(X) = E(X^2) - [E(X)]^2$
- $D(X+Y) = D(X) + D(Y)$

那么

$$
\begin{aligned}
&D(XY)\\
&=E[XY-E[XY]]^2\\
&=E[X^2Y^2-2XYE[XY]+E^2[XY]]\\
&=E[X^2]E[Y^2]-2E^2[X]E^2[Y]+E^2[X]E^2[Y]\\
&=E[X^2]E[Y^2]-E^2[X]E^2[Y]\\
&=(D[X]+(E[X])^2)\ (D[Y]+(E[Y])^2)-E^2[X]E^2[Y]\\
&=D[X]D[Y] + D[X](E[Y])^2 + D[Y](E[X])^2 + E^2[X]E^2[Y] - E^2[X]E^2[Y]\\
&=D[X]D[Y] + D[X](E[Y])^2 + D[Y](E[X])^2
\end{aligned}
$$

$E(X) = 0, E(Y) = 0$, 则$D(XY) = D(X)D(Y)$

我们看看神经网络里面每一层输出的方差计算：



还是这个网络，我们看第一层第一个神经元的方差应该怎么算： 这里我们假设输入数据和权重都初始化的均值为0，方差为1的标准正态。

$$
\begin{aligned}
H_{11} &= \sum_{i=0}^{n} X_i * W_{1i} \\
D(H_{11}) &= \sum_{i=0}^{n} D(X_i) * D(W_{1i}) \\
&= n * (1 * 1) \\
&= n \\
\mathrm{std}(H_{11}) &= \sqrt{D(H_{11})} = \sqrt{n}
\end{aligned}
$$

- 假设输入数据和权重都初始化的均值为0，方差为1的标准正态。
- 这样经过一个网络层就发现方差扩大了 n 倍。
- 而我们上面用了100个网络层， 那么这个方差会指数增长，所以我们后面才会出现输出层方差nan的情况。

那么我们怎么解决这种情况呢?

- 很简单，让网络层的输出方差保持尺度不变-->怎么做呢?
- 分析一下网络层的输出方差:

$$
\begin{aligned}
D(H_{11}) &= \sum_{i=0} D(X_i) * D(W_{1i}) \\
&= n * (1 * 1) \\
&= n
\end{aligned}
$$

- 发现，每一层的输出方差决定于：每一层神经元个数，前一层输出方差，本层权重的方差
- 如果想让方差的尺度不变，就是让每一层输出方差都是1， 也就是 D(H11)=1
- 这样后面多层相乘，那么也不会变这个尺度。

具体做法

- 我们把权重的方差 $D(W)$ 当作需要求解的未知数。

$$D(H_1) = n \cdot D(X) \cdot D(W)$$

我们设定我们的理想目标 $D(H_1) = 1$，并且我们知道输入方差 $D(X) = 1$：

$$1 = n \cdot 1 \cdot D(W)$$

求解 $D(W)$，我们就得到了解决方案：

$$D(W) = \frac{1}{n}$$

- 结论： **要想让输出方差保持稳定，权重的方差必须与输入神经元的数量成反比。**

在上面代码中改一句话:

```
    def initialize(self):
        # 遍历模型的子模块
        for m in self.modules():
            if isinstance(m, nn.Linear):
                nn.init.normal_(m.weight.dat
```

```
61
62      net = MLP(neural_nums, layer_nums)
63      net.initialize()
64
65      inputs = torch.randn((batch_size, neural_nums))
66
67      output = net(inputs)
68      print(output)
69
70  # ========================================== calculate
71
72  # flag = 0
73  flag = 1
74
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS VARIABLES

```
layer:27, std:0.9534041881561279
layer:28, std:0.9811319708824158
layer:29, std:0.9953019022941589
layer:30, std:0.9773916006088257
layer:31, std:0.9655940532684326
layer:32, std:0.9270440936088562
layer:33, std:0.9329946637153625
layer:34, std:0.9311841726303101
layer:35, std:0.9354336261749268
layer:36, std:0.9492132067680359
layer:37, std:0.9679954648017883
layer:38, std:0.9849981665611267
layer:39, std:0.9982335567474365
```



所以我们只要采用恰当的权值初始化方法，就可以实现多层神经网络的输出值的尺度维持在一定范围内,这样在反向传播的时候，就有利于缓解梯度消失或者爆炸现象的发生

上面的网络只是一个线性网络，在实际中我们还得考虑激活函数的存在 我们从上面的前向传播中加一个激活函数tanh()再看一下结果：

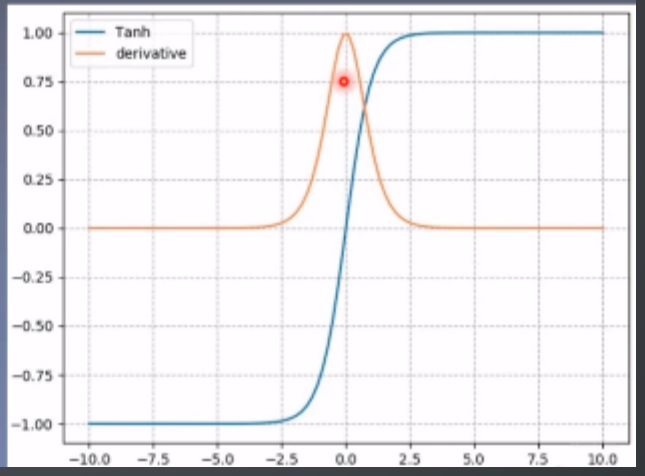
nn.tanh

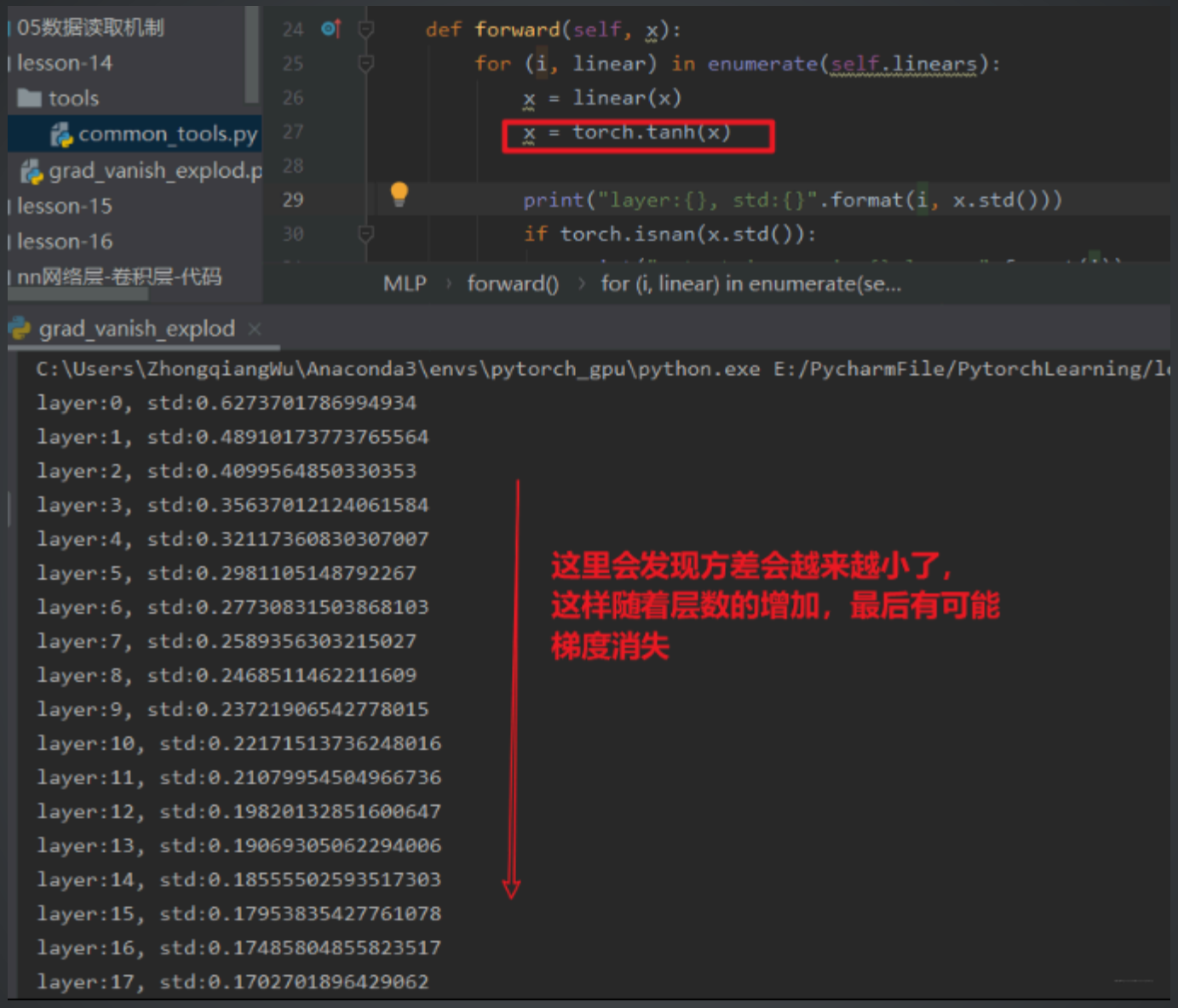
计算公式：$y = \frac{\sin x}{\cos x} = \frac{e^x - e^{-x}}{e^{-} + e^{-x}} = \frac{2}{1+e^{-2x}} + 1$

梯度公式：$y' = 1 - y^2$

特性：

- 输出值在(-1,1)，数据符合0均值
- 导数范围是(0, 1),易导致梯度消失





那么，具有激活函数的时候，怎么对权重进行初始化呢?

2.2 Xavier初始化

方差一致性：保持数据尺度范围维持在恰当范围， 通常方差为1。 如果有了激活函数之后，我们应该怎么对权重初始化呢?

2010年Xavier发表了一篇文章，详细探讨了如果有激活函数的时候，如何进行权重初始化， 当然它也是运用的方差一致性原则， 但是它这里**考虑的是饱和激活函数**， 如sigmoid，tanh。

Xavier文章中有个这样的公式，对于一层的权重W的方差D(W) 做了折中的设想，即D(W)要满足：

$$D(W) = \frac{2}{n_{in}+n_{out}}$$

其中，n_{in} 是输入神经元的数量，n_{out} 是输出神经元的数量。

- 第一步：我们现在的目标是，找到一个均匀分布的边界 [-a，a]，使得一层权重的方差正好等于上面这个值 10。
- 一个在 [-a, a] 区间内均匀分布的随机变量（就是这一层权重），其方差的计算公式是：

$$D(W) = \frac{(a-(-a))^2}{12} = \frac{(2a)^2}{12} = \frac{4a^2}{12} = \frac{a^2}{3}$$

第二步：求解 'a'

现在，我们让两个表示 $D(W)$ 的表达式相等，然后解出 a。

$$\frac{a^2}{3} = \frac{2}{n_{in}+n_{out}}$$

两边同时乘以3：

$$a = \frac{6}{n_{in}+n_{out}}$$

两边同时开方：

$$a = \sqrt{\frac{6}{n_{in}+n_{out}}} = \frac{\sqrt{6}}{\sqrt{n_{in}+n_{out}}}$$

- 这样，我们就得到了Xavier均匀分布初始化的最终取值范围。 这就是Xavier初始化方法

那么在代码中怎么用呢？还是上面的那个代码例子，我们在参数初始化里面用Xavier初始化权重：

```
    def initialize(self):
        for m in self.modules():
            if isinstance(m, nn.Linear):
                # Xavier初始化权重
                tanh_gain = nn.init.calculat
                nn.init.xavier_uniform_(m.we
```

这里面用到了一个函数

`nn.init.calculate_gain(nonlinearity, param=None)`

这个函数的作用是计算激活函数的**方差变化尺度**

- 其实就是输入数据的方差除以经过激活函数之后的输出数据的方差。
- nonlinearity表示激活函数的名称，如`tanh`, param表示激活函数的参数，如Leaky ReLU的`negative_slop`。（这里不用也行，但得知道这个方法）。

- 这时候再来看一下最后的结果：

bigAI-03-06-20241125.py × launch.json

01_PyTorch_Practice > lesson4 > bigAI-03-06-20241125.py > ...

```
65
66      net = MLP(neural_nums, layer_nums)
67      net.initialize()
68
69      inputs = torch.randn((batch_size, neural_nums))  # normal: mean=0, std=1
70
71      output = net(inputs)
72      print(output)
73
74  # ========================================== calculate gain ======================
75
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS VARIABLES

```
layer:91, std:0.6497945785522461
layer:92, std:0.6458892226219177
layer:93, std:0.6458885669708252
layer:94, std:0.6530362963676453
layer:95, std:0.6515855193138123
layer:96, std:0.643466055393219
layer:97, std:0.6426210403442383
layer:98, std:0.6407480835914612
layer:99, std:0.6442216038703918
```

所以Xavier权重初始化，有利于缓解带有sigmoid，tanh的这样的饱和激活函数的神经网络的梯度消失和爆炸现象。

数relu

而Xavier初始化对于relu就不好使了，不信我们看看：

bigAI-03-06.py × launch.json

01_PyTorch_Practice > lesson4 > bigAI-03-06.py > ...

```
13  class MLP(nn.Module):
19      def forward(self, x):
20          for (i, linear) in enumerate(self.linears):
21              x = linear(x)
22              # x = torch.tanh(x)
23              x = torch.relu(x)
24
25              print("layer:{}, std:{}".format(i, x.std()))
26              if torch.isnan(x.std()):
27                  print("output is nan in {} layers".format(i))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS VARIABLES

```
layer:89, std:1783507.25
layer:90, std:2239068.0
layer:90, std:2239068.0
layer:91, std:2429546.0
layer:91, std:2429546.0
layer:92, std:2928562.0
layer:93, std:2883037.75
layer:94, std:3230928.75
layer:95, std:3661650.75
layer:96, std:4741352.0
layer:97, std:5300345.0
layer:98, std:6797732.0
layer:99, std:7640650.0
```

2.3 Kaiming初始化

- Kaiming初始化 (专为ReLU设计)
- **内容:**
  - 现在，ReLU是比tanh更流行的激活函数。但如果我们把Xavier方法用在ReLU上，会发现它又失效了，数据波动范围会越来越大，最终导致梯度爆炸。
  - **为什么？** 因为ReLU函数有个特点：它会把所有负数都变成0。这意味着，网络中大约一半的神经元会“死亡”(输出为0)，这改变了数据的分布和方差。
  - **Kaiming登场：** 2015年，何恺明（Kaiming He）等人针对ReLU及其变种，提出了一种新的初始化方法。

1. Kaiming推导的一层方差公式 $D(W) = \frac{2}{n_i}$

- 这里的 $D(W)$ 表示权重 W 的方差。
- n_i 是当前层的输入神经元数量（也称为 fan-in）。

2. 修正后的方差公式 $D(W) = \frac{2}{(1+a^2)+n_i}$

- 这是对权重方差的进一步修正，考虑了激活函数的影响。
- a 是激活函数的负斜率（例如 Leaky ReLU 中负斜率参数）。
- 该公式用于生成服从正态分布或均匀分布的初始权重。

那么Kaiming初始化权重方法怎么用呢?

```
def initialize(self):
    for m in self.modules():
        if isinstance(m, nn.Linear):
            nn.init.kaiming_normal_(m.we
            # nn.init.normal_(m.weight.d
```

我们可以看一下结果：

bigAI-03-06.py × launch.json

01_PyTorch_Practice > lesson4 > bigAI-03-06.py > ...

```
13    class MLP(nn.Module):
32        def initialize(self):
52                    # nn.init.xavier_uniform_(m.weight.data, gain=relu_gain)
53
54                    # relu 激活函数
55                    # nn.init.normal_(m.weight.data, std=np.sqrt(1 / self.neural_num))
56                    nn.init.kaiming_normal_(m.weight.data)
57
58    # ================================== print output ==============================
59    # flag = 0
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS VARIABLES

```
layer:92, std:0.6094604730606079
layer:91, std:0.6606621742248535
layer:92, std:0.6094604730606079
layer:92, std:0.6094604730606079
layer:93, std:0.6019102334976196
layer:94, std:0.595421552658081
layer:95, std:0.6624555587768555
layer:96, std:0.6377885341644287
layer:97, std:0.6079285740852356
layer:98, std:0.6579315066337585
layer:99, std:0.6668476462364197
```

权重初始化方法总结

- **核心要点：** 不恰当的权重初始化会导致梯度消失或爆炸，让模型无法训练。
- **选择原则：** 根据你的激活函数来选择合适的初始化方法。
    - 如果使用 **`sigmoid` 或 `tanh`**，优先考虑 **Xavier** 初始化。
    - 如果使用 **ReLU 或其变体**（LeakyReLU等），优先考虑 **Kaiming** 初始化。
- PyTorch还提供了很多其他方法，如均匀分布、正态分布、常数初始化等，可以根据特定需求选用。
- 到此，关于模型“起点”的话题就结束了。

3. 损失函数

- **损失函数是干嘛的？** 它是一个**衡量“理想”与“现实”差距**的函数。在机器学习中，就是衡量模型的**预测值** 和真实的 **标签值**之间的差异。
- **它的作用：** 损失函数会计算出一个“损失值”(或叫“误差”、“成本”)。这个值越大，说明模型预测得越差。
- **训练的目标：** 整个模型训练的过程，就是想尽一切办法（通过优化器调整权重），把这个“损失值”降到最低。
- **在PyTorch中：** 损失函数本身也是一个Module，和我们之前学的网络层一样，使用起来非常方便。最常用的分类损失函数是nn.CrossEntropyLoss。

3.1损失函数初步介绍

损失函数： 衡量模型输出与真实标签的差异。

- 我们下面看一下Pytorch中的损失函数的真实面目：

```
class _Loss(Module):
    def __init__(self, size_average=None, reduce=None,
                 reduction='mean'):
        super(_Loss, self).__init__()
        if size_average is not None or reduce is not None:
            self.reduction = _Reduction.legacy_get_string(
                                size_average, reduce)
        else:
            self.reduction = reduction
```

即将舍弃，不用学这俩参数

原来_Loss也是继承于Module

- 之前人民币二分类的例子里面有交叉熵损失函数
- 我们在定义损失函数和使用损失函数的地方打上断点，并且开始debug：

launch.json | bigAI-03-03.py

01_PyTorch_Practice > bigAI-03-03.py > ...

```
71    criterion = nn.CrossEntropyLoss()
72
73    # ============================ step 4/5 优化器 ============================
74    optimizer = optim.SGD(net.parameters(), lr=LR, momentum=0.9)
75    scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=10, gamma=0.1)
76
77    # ============================ step 5/5 训练 ============================
78    train_curve = list()
79    valid_curve = list()
80
81    for epoch in range(MAX_EPOCH):
82
83        loss_mean = 0.
84        correct = 0.
85        total = 0.
86
87        net.train()
88        for i, data in enumerate(train_loader):
89
90            # forward
91            inputs, labels = data
92            outputs = net(inputs)
93
94            # backward
95            optimizer.zero_grad()
96            loss = criterion(outputs, labels)
97            loss.backward()
```

- 下面我们就详细的学习一个重要的函数，也是上面例子里面的函数nn.CrossEntropyLoss
- 这个在分类任务中很常用，所以下面得详细的说说。

nn.CrossEntropyLoss: 进行交叉熵计算。

```
nn.CrossEntropyLoss(weight=None,
                    size_average=None,
                    ignore_index=-100,
                    reduce=None,
                    reduction='mean')
```

3.2交叉熵：从“信息”说起

- weight: 当你的数据里某个类别特别多或特别少时（类别不均衡），可以用这个参数给数量少的类别**增加“话语权”**（更高的权重），让模型更重视它。
- ignore_index: 忽略某个类别
- reduction：决定如何汇总所有样本的损失。'mean'表示求平均（最常用），'sum'表示求和。

首先从熵开始

- 熵是信息论之父香农从热力学借鉴来的名词，用来描述事件的不确定性
- 一个事物不确定性越大，熵就越大。- 比如明天会下雨这个熵就比明天太阳从东边升起这个熵要大
- 熵的公式： 信息熵 $H(P)$ 是用来衡量随机变量 X 的不确定性或信息量的一个指标。公式为：

$$H(P) = \mathbb{E}_{X\sim P}[I(x)] = -\sum_{i=1}^{N} P(x_i)\log P(x_i)$$

其中：

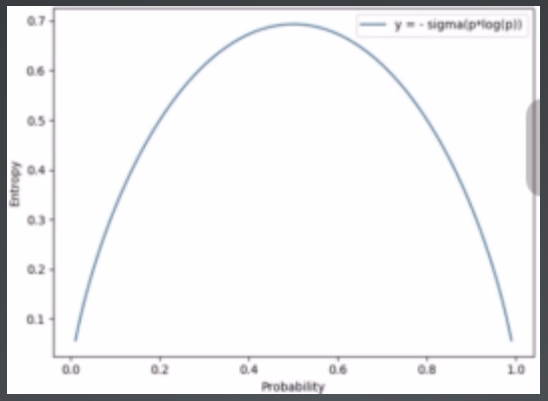
- $P(x_i)$：表示随机变量 X 取值为 x_i 的概率。
- $I(x) = -\log P(x)$：称为信息量，表示事件 x 的自信息。
- $H(P)$：是对所有可能事件的加权平均信息量。 从上式可知，熵是自信息$I(x)$的一个期望

先看看自信息是什么？下面是自信息的公式：

$$I(x) = -\log P(x)$$

- 这个公式说明，自信息就是一个事件发生的概率，然后取对数再取反。
- 也就是一个事件如果发生的概率越大，那么自信息就会少。
- 所有事件发生的概率都很大，那么熵就会小，则事件的不确定性就小。
- 即发生该事件所携带的信息量

看个图就好理解了：



- 这是一个两点分布的一个信息熵。可以看到，当概率是0.5的时候熵最大。也就是事件的不确定性最大，熵大约是0.69
- 这个在二分类模型中经常会碰到，模型训练坏了的时候，或者刚训练的时候，我们就会发现分类Loss值(信息熵)也可能是0.69
- 这时候就说模型目前输出随机，即没有判断能力
- 优化模型，就是要使其输出确定性高，分类信息熵最小

相对熵

$$D_{KL}(P\|Q) = \mathbb{E}_{x\sim P}\left[\log \frac{P(x)}{Q(x)}\right]$$

- 用来衡量两个概率分布 P 和 Q 之间“差异”，也就是两个分布之间的距离。
  - 当 $P = Q$ 时，$D_{KL}(P\|Q) = 0$，表示两者完全一致。
  - 如果 $P \neq Q$，则 $D_{KL}(P\|Q) > 0$，值越大表示两者差异越大。
- 这里的P是数据的真实分布，
- Q是神经网络模型判断输出的分布，
- 这里就是用Q的分布去逼近P的分布。所以这不具备对称性

交叉熵

- 公式如下：

$$\mathrm{H}(\boldsymbol{P},\boldsymbol{Q})=-\sum_{\mathrm{i}=1}^{\mathrm{N}}\boldsymbol{P}\left(\boldsymbol{x}_{\mathrm{i}}\right)\log\boldsymbol{Q}\left(\boldsymbol{x}_{\mathrm{i}}\right)$$

- 其实**交叉熵=信息熵+相对熵**，这个关系可以把相对熵化简一下：

$$\begin{aligned}\boldsymbol{D}_{\mathrm{KL}}(\boldsymbol{P},\boldsymbol{Q}) &= \boldsymbol{E}_{\mathrm{x}\sim\mathrm{p}}\left[\log\frac{\boldsymbol{P}(\boldsymbol{x})}{\mathrm{Q}(\boldsymbol{x})}\right]\\ &= \boldsymbol{E}_{\boldsymbol{x}\sim\boldsymbol{p}}[\log\boldsymbol{P}(\boldsymbol{x})-\log\boldsymbol{Q}(\boldsymbol{x})]\\ &= \sum_{\mathrm{i}=1}^{\mathrm{N}}\boldsymbol{P}\left(\boldsymbol{x}_{\mathrm{i}}\right)\left[\log\boldsymbol{P}\left(\boldsymbol{x}_{\mathrm{i}}\right)-\log\boldsymbol{Q}\left(\boldsymbol{x}_{\mathrm{i}}\right)\right]\\ &= \sum_{\mathrm{i}=1}^{\mathrm{N}}\boldsymbol{P}\left(\boldsymbol{x}_{\mathrm{i}}\right)\log\boldsymbol{P}\left(\boldsymbol{x}_{\mathrm{i}}\right)-\sum_{\mathrm{i}=1}^{\mathrm{N}}\boldsymbol{P}\left(\boldsymbol{x}_{\mathrm{i}}\right)\log\boldsymbol{Q}\left(\boldsymbol{x}_{\mathrm{i}}\right)\\ &= -\text{信息熵}+\text{交叉熵}\end{aligned}$$

- 因此**交叉熵=信息熵+相对熵**

根据上面的推导我们得到：

$$\mathrm{H}(\boldsymbol{P}, \boldsymbol{Q}) = \boldsymbol{D}_{\mathrm{KL}}(\boldsymbol{P}, \boldsymbol{Q}) + \mathrm{H}(\boldsymbol{P})$$

- 因为我们训练集取出来之后就是固定的了，即数据的真实分布确定，数据的信息熵H(P)就是一个常数
- 所以，在机器学习模型中，我们最小化交叉熵，其实就是最小化相对熵

- Pytorch对样本集的分类预测的交叉熵损失函数是怎么计算的，公式如下：

$$L = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{K} y_{ic}\log(p_{ic})$$

含义：

- 这是机器学习中常用的**交叉熵损失函数**，用于分类任务。
- 它衡量模型预测的概率分布 p_{ic} 与真实分布 y_{ic} 之间的差异。

符号解释：

- N：样本总数。
- K：类别总数。
- p_{ic}：第 i 个样本属于类别 c 的预测概率。
- y_{ic}：第 i 个样本属于类别 c 的真实标签（one-hot 编码，只有一个值为 1，其余为 0）。

one-hot 编码的作用：

- 对于每个样本，真实类别对应的 $y_{ic} = 1$，其余类别对应的 $y_{ic} = 0$。
- 因此，对于每个样本，只有真实类别的预测概率会参与计算，其余项被忽略。

以单个样本的预测的交叉熵为例

$$L_i = \sum_{c=1}^{K} y_{ic} log(p_{ic})$$

那么3个样本

$$L_1 = 0 * log(0.2) + 1 * log(0.3) + 0 * log(0.5) = -1.2039$$

$$L_2 = 1 * log(0.3) + 0 * log(0.2) + 0 * log(0.5) = -1.2039$$

$$L_3 = 0 * log(0.4) + 0 * log(0.4) + 0 * log(0.2) = -1.6094$$

- 最终的交叉熵只不过是做了N这样的计算，然后平均一下，加个负号：

$$L = -\frac{1}{3}(L_1 + L_2 + L_3) = 1.3391$$

这里的 $p_{ic}, c = 1, 2, 3$

之和为1。没错，这是网络的输出做了softmax后得到的结果。

Ref： 一文详解PyTorch中的交叉熵 - 知乎 (zhihu.com)

nn.CrossEntropyLoss: 进行交叉熵计算。

```
nn.CrossEntropyLoss(weight=None,
                    size_average=None,
                    ignore_index=-100,
                    reduce=None,
                    reduction='mean')
```

- weight：当你的数据里某个类别特别多或特别少时（类别不均衡），可以用这个参数给数量少的类别**增加“话语权”**（更高的权重），让模型更重视它。
- ignore_index: 忽略某个类别
- reduction：决定如何汇总所有样本的损失。'mean'表示求平均（最常用），'sum'表示求和。

随堂作业

信息论中，_____用来衡量一个事件的不确定性或混乱程度，一个均匀的硬币（正反概率均为0.5）拥有最大的不确定性，因此它的这个值也最大。

答案: 熵

- **解析**: 熵是事件不确定性的度量。当所有可能性相等时，不确定性最高，熵也最大。

3.2.1还有几个交叉熵损失函数的特例

1. nn.BCELoss

- 这是交叉熵的一个特例，专门用于只有**两个类别**的场景（比如：是/不是，垃圾/非垃圾邮件）。
- 注意：使用它之前，需要手动用Sigmoid函数将模型的输出压缩到0-1之间。

```
nn.BCELoss(weight=None,
    size_average=None,
    reduce=None,
    reduction='mean')
```

这里的参数和上面的一样，看看这个计算公式：

$$L = -\frac{1}{N}\sum_{i=1}^{N}[y_i log(p_i) + (1-y_i) log(1-p_i)]$$

2. nn.BCEWithLogitsLoss

- 这是BCELoss的升级版，它**内置了Sigmoid**。
- **优点：** 使用起来更方便，而且数值计算上更稳定。**推荐在二分类任务中使用这个。**

```
nn.BCEWithLogitsLoss(weight=None,
                     size_average=None,
                     reduce=None, reduction='mean',
                     pos_weight=None)
```

- 这里的参数多了一个pow_weight,这个是平衡正负样本的权值用的， 对正样本进行一个权值设定。
- 比如我们正样本有100个，负样本有300个，那么这个数可以设置为3，在类别不平衡的时候可以用。

3.3 剩余的其他损失函数介绍

1. nn.L1Loss (或MAE Loss): 计算**绝对误差** $|预测值 - 真实值|$。对异常值不那么敏感。

nn.L1Loss(size_average=None, reduce=None, reduction='mean')

$$l_n = |x_n - y_n|$$

2. nn.MSE (或L2 Loss): 计算**均方误差** $|\textit{预测值} - \textit{真实值}|^2$。对误差大的点惩罚更重。

nn.MSELoss(size_average=None, reduce=None, reduction='mean')

$$l_n = (x_n - y_n)^2$$

3. nn.SmoothL1Loss

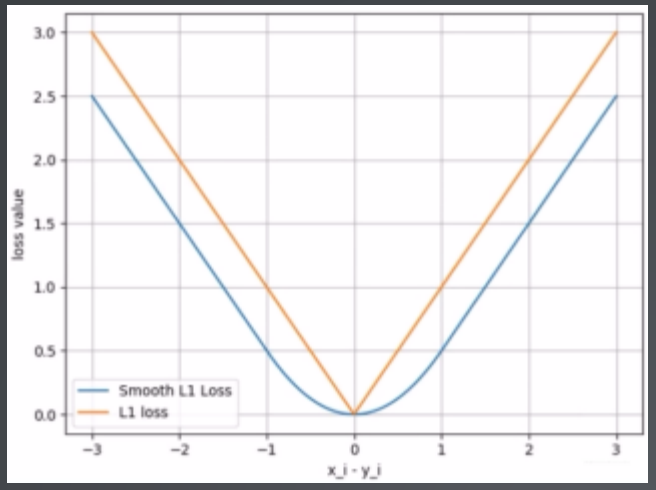
结合了L1和L2的优点，在误差较小时像MSE，在误差较大时像L1，更加鲁棒。

nn.SmoothL1Loss(size_average=None, reduce=None, reduction='mean')

$$\text{loss}(x, y) = \frac{1}{n}\sum_i z_i$$

$$z_i = \begin{cases} 0.5(x_i - y_i)^2, & \text{if } |x_i - y_i| < 1 \\ |x_i - y_i| - 0.5, & \text{otherwise} \end{cases}$$

那么这个平滑到底是怎么体现的呢?



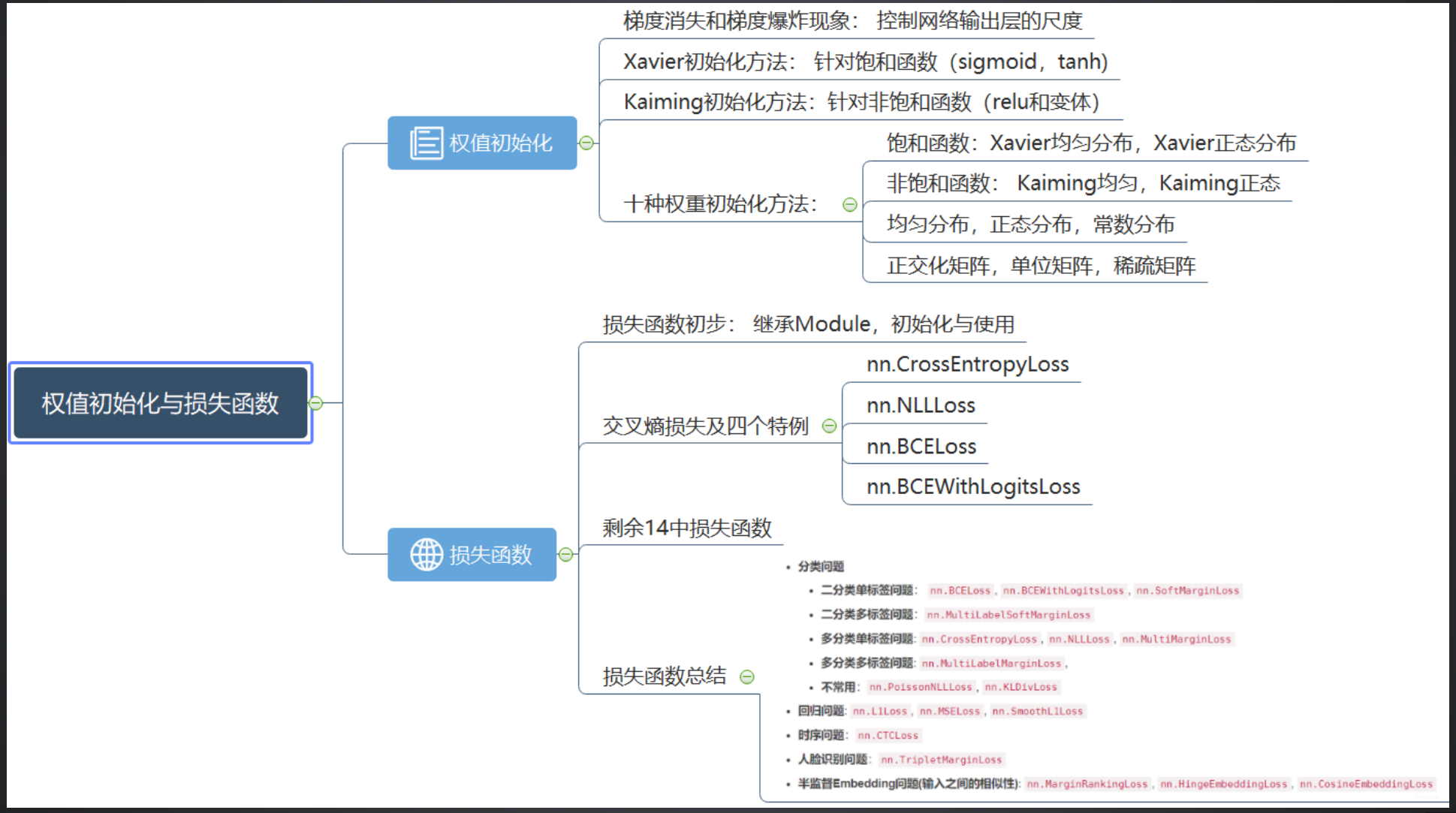
采用这种平滑的损失函数可以减轻离群点带来的影响。

- **还有很多其他任务的损失函数：**
- 比如用于人脸识别的TripletMarginLoss，
- 用于时序问题的CTCLoss等等。
- 我们不需要全部记住，但要知道，选择合适的损失函数是解决特定问题的关键一步。

Ref:
https://blog.csdn.net/wuzhongqiang/article/details/105590118

4. 总结

下面依然是一个思维导图把知识拎起来，方便后面的速查：



本章要点:

1. 权重初始化 (设置好起点):

- 是防止梯度消失/爆炸，保证模型能顺利训练的关键。
- 核心法则：根据激活函数选择方法（ReLU用Kaiming, tanh用Xavier）。

2. 损失函数 (选好裁判):

- 是模型训练的“指挥棒”，告诉模型优化的方向。
- 核心法则：根据任务类型选择函数（分类用交叉熵, 回归用MSE/L1）。